Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Automatică și Calculatoare
Departamentul Calculatoare și Tehnologia Informației

Computer Architecture
Laborator 11: Calculator Didactic

Ștefan-Dan Ciocîrlan · 2026-2027

Cuprins

1. Moduri de adresare 1
   1.1. Operandul nu se găseşte în memorie 2
   1.2. Operandul e specificat doar prin deplasament 2
   1.3. Operandul e specificat doar prin registre 2
   1.4. Operandul e specificat prin registre şi deplasament 3
   1.5. Calculatorul didactic 3

1. Moduri de adresare

Modurile de adresare reprezintă modalitatea prin care se poate specifica adresa efectivă a operanzilor. Instrucțiunile calculatorului didactic pot prelucra maxim doi operanzi ce se pot găsi:

- Ambii în registrele generale RG;
- Unul în registrele generale RG şi altul în memorie;
- Unul în registrele generale RG şi altul în cadrul instrucțiunii respective (numit operand imediat);
- Unul în memorie şi altul imediat.

Modurile de adresare permise de calculatorul didactic sunt tipice arhitecturilor CISC, fiind derivate din arhitectura standard x86. În total, procesorul permite 11 moduri de adresare, ceea ce îi conferă o foarte bună flexibilitate în programare. Pentru o mai ușoară înțelegere, le vom structura astfel:

Specificarea operandului	**Moduri de adresare**
Operandul nu se găseşte în memorie	Adresare directă la registru, Adresare imediată
Operandul e specificat doar prin deplasament	Adresare directă, Adresare indirectă
Operandul e specificat doar prin registre	Adresare indirectă prin registru, Adresare indirectă prin suma de registre, Adresare indirectă prin suma de registre cu autoincrementare, Adresare indirectă prin suma de registre cu autodecrementare
Operandul e specificat prin registre şi deplasament	Adresare Bazată, Adresare Indexată, Adresare Bazată Indexată

În continuare vor fi prezentate şi discutate aceste moduri de adresare.

1.1. Operandul nu se găseşte în memorie

1.1.1. 1. Adresare directă la registru

Operandul se găseşte în RG.

Exemplu: `MOV RA, RB`

Instrucțiunea are ca efect încărcarea în registrul RA a valorii din registrul RB.

1.1.2. 2. Adresare imediată

Operandul este specificat în instrucțiune.

Exemplu: `MOV RA, 7`

Instrucțiunea va avea ca efect încarcarea valorii „7” în registrul RA. „7” poartă numele de operand imediat.

1.2. Operandul e specificat doar prin deplasament

1.2.1. 3. Adresare directă

Adresa efectivă este specificată în instrucțiune.

Exemplu: `MOV RA, [12]`

Instrucțiunea va încărca valorea aflată în memorie la adresa „12” în registrul RA. „12” poarta numele de deplasament.

1.2.2. 4. Adresare indirectă

Adresa efectivă se citește din memorie, din locația a cărei adresă este specificată în instrucțiune.

Exemplu: `MOV RA, [[12]]`

Instrucțiunea are ca efect încărcarea valorii aflată la adresa ce se găsește în memorie la adresa „12”. Acest mod de adresare seamană foarte bine cu pointerii din C. La adresa „12” se găsește pointer-ul către operand.

1.3. Operandul e specificat doar prin registre

1.3.1. 5. Adresare indirectă prin registru

Adresa efectivă se găsește în unul din registrele XA, XB, BA, BB.

Exemplu: `MOV RA, [BA]`

Instrucțiunea va încărca în RA valoarea aflată în memorie la adresa conținută în BA.

1.3.2. 6. Adresare indirectă prin sumă de registre

Adresa efectivă se obține ca sumă a conținutului unui registru de bază cu conținutul unui registru index.

Exemplu: `MOV RA, [BA][XA]` sau `MOV RA, [BA+XA]`

Instrucțiunile încarcă în RA valoarea aflată în memorie la adresa obținută prin adunarea conținutului registrelor BA şi XA. BA, BB poartă numele de registre de bază, iar XA, XB poartă numele de registre index. Întotdeauna suma se va face între un registru bază şi unul index.

1.3.3. 7. Adresare indirectă prin sumă de registre cu autoincrementare

Față de modul precedent de adresare apare deosebirea că registrele index se incrementează după generarea adresei efective. Incrementarea registrului index (XA sau XB) are deci loc după participarea la calculul adresei efective.

Exemplu: `MOV RA, [BA][XA+]` sau `MOV RA, [BA+XA+]`

Aceste instrucțiuni sunt echivalente cu:

```
MOV RA, [BA][XA]
INC XA
```

1.3.4. 8. Adresare indirectă prin sumă de registre cu autodecrementare

Adresa efectivă se obține prin suma unui registru de bază cu continutul registrului XA. Înaintea generării adresei, registrul XA este decrementat.

Exemplu: `MOV RA, [BA][XA-]` sau `MOV RA, [BA+XA-]`

Aceste instrucțiuni sunt echivalente cu:

```
DEC XA
MOV RA, [BA][XA]
```

Observații: Motivul pentru care XB nu poate fi utilizat în acest mod de adresare nu este unul logic, ci unul tehnic: biții din codul de instrucțiune nu au fost suficienți pentru a se putea codifica și acest mod.

1.4. Operandul e specificat prin registre și deplasament

1.4.1. 9. Adresare bazată

Adresa efectivă se obține prin adunarea unui registru de bază cu un deplasament.

Exemplu: `MOV RA, [BA]+7` sau `MOV RA, [BA+7]`

Instrucțiunea încarcă în registrul RA valoarea aflată în memorie la adresa rezultată în urma adunării conținutului registrului de bază cu deplasamentul (7).

1.4.2. 10. Adresare indexată

Adresa efectivă se obține prin adunarea unui registru index cu un deplasament.

Exemplu: `MOV RA, [XA]+7` sau `MOV RA, [XA+7]`

Instrucțiunea încarcă în registrul RA valoarea aflată în memorie la adresa rezultată în urma adunării conținutului registrului de index cu deplasamentul (7).

1.4.3. 11. Adresare bazată indexată

Adresa efectivă se obține prin adunarea unui registru de bază cu un registru index și cu un deplasament.

Exemplu: `MOV RA, [BA][XA]+7` sau `MOV RA, [BA+XA+7]`

Instrucțiunea încarcă în registrul RA valoarea aflată în memorie la adresa rezultată în urma adunării conținutului registrului de bază (BA) cu registrul index (XA) și cu deplasamentul (7).

1.5. Calculatorul didactic

Procesorul calculatorului didactic suportă mai multe moduri de adresare a operanzilor. Până acum am tratat doar cazul operanzilor aflați în registre (adresare directă la registru), identificat prin câmpul *MOD* având valoarea *11*. Cu acest mod de adresare suntem însă limitați la a lucra numai

cu cele 8 registre generale. Pentru a putea lucra și cu date aflate în memoria RAM, procesorul calculatorului didactic oferă o serie de alte moduri de încărcare a operanzilor.

Încărcarea unui operand din memoria RAM necesită determinarea adresei la care se află stocat acel operand. Aceasta poate fi stocată direct în instrucțiune, cum se întâmplă în cazul adresării directe, unde instrucțiunea este formată din 2 cuvinte (pe 16 biți), iar al doilea cuvânt conține valoarea adresei (numită și deplasament), sau poate fi calculată folosind diferite combinații de registre cu/fără deplasament. Calculul se face prin adunarea valorilor curente ale unor registre (nu toate combinațiile sunt suportate) și folosirea valorii rezultate ca o adresă pentru memorie.

1.5.1. Adresarea indirectă prin registru

Adresa efectivă se găsește într-unul din registrele *XA*, *XB*, *BA* sau *BB*. Modul de determinare a acesteia este prezentat în imaginea de mai jos.

Ex: `mov RA, [BA]`

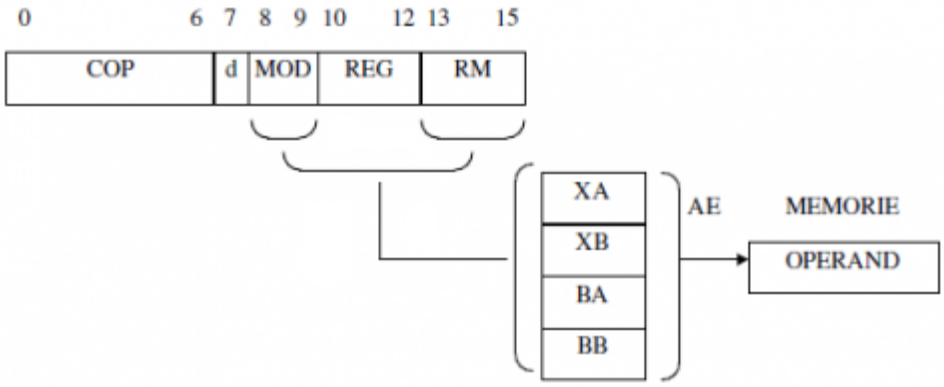


Figura 1: Adresare indirectă prin registru

1.5.2. Adresarea indirectă prin sumă de registre

Adresa efectivă se obține prin sumarea conținutului unui registru de bază (*BA* sau *BB*) cu conținutul unui registru index (*XA* sau *XB*). Modul de determinare a acesteia este prezentat în imaginea de mai jos.

Ex: `mov RA, [BA + XA]`

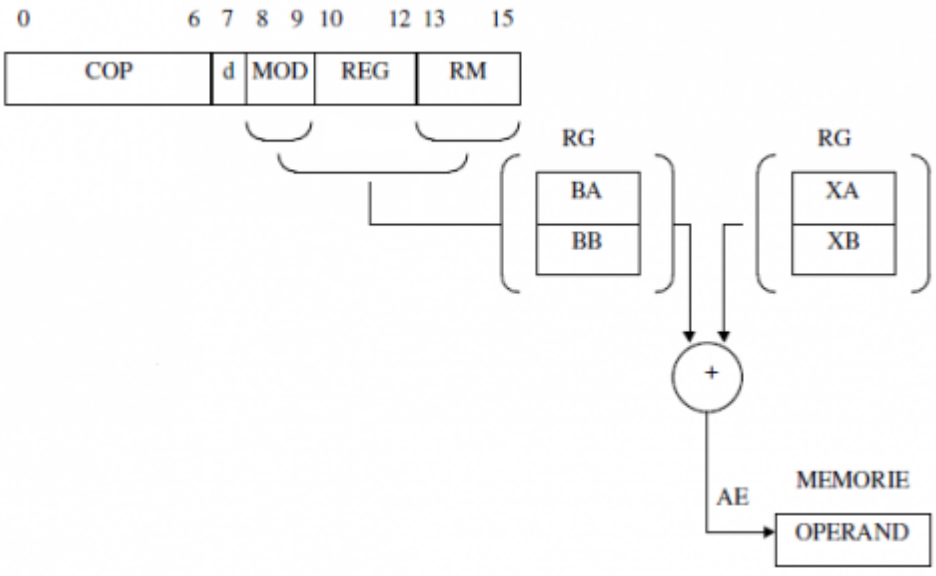


Figura 2: Adresare indirectă prin sumă de registre

1.5.3. Implementare

Automatul care trebuie implementat în UC este descris în diagrama de stări din figura de mai jos.

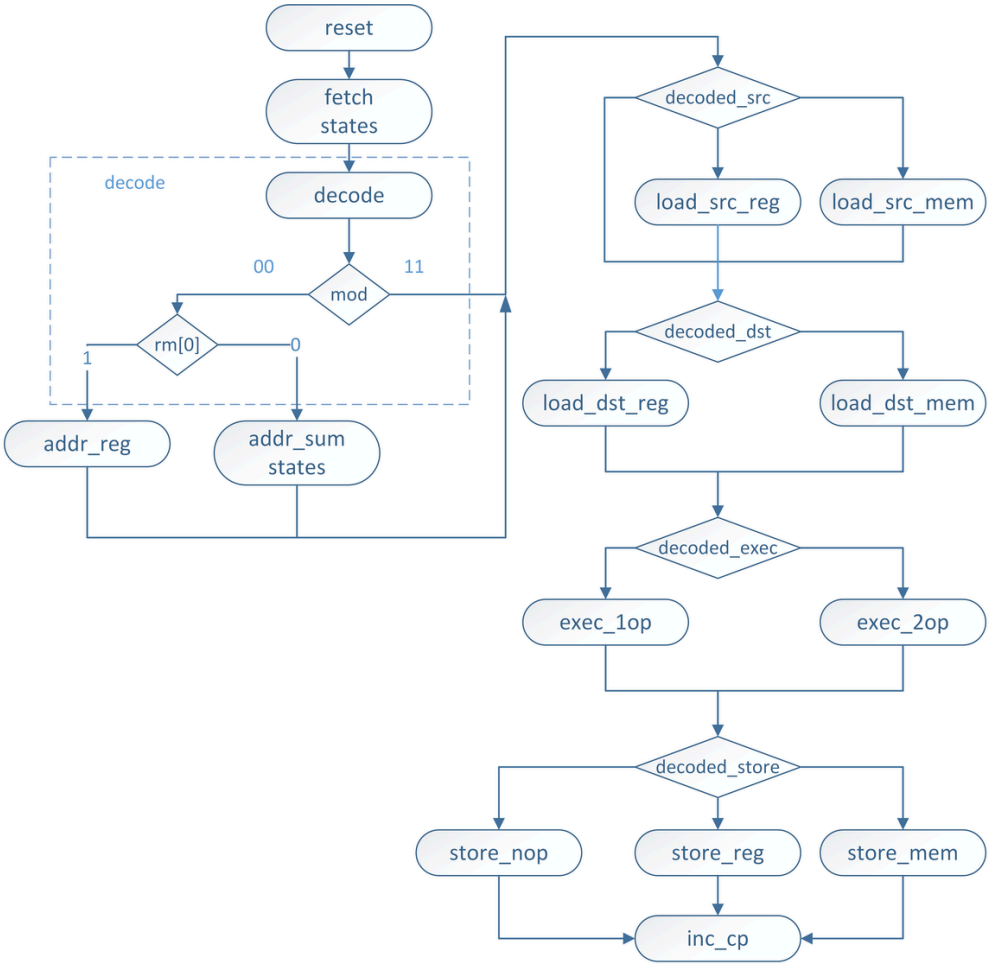


Figura 3: Diagrama de stări a unității de comandă

După cum puteți observa, la stările deja implementate în laboratoarele anterioare au fost adăugate o serie de stări noi, necesare pentru adresarea indirectă prin registru sau sumă de registre. În cadrul acestui laborator va trebui să implementați aceste noi stări și să completați condițiile corecte pentru decodificarea noilor moduri de adresare.

TODO: schimbat total cu cram.data

Memoria calculatorului didactic este implementată cu ajutorul unui IP Core Xilinx de tipul Block Memory. Parametrii unui core sunt stocați într-un fișier *.xco*. Acest fișier este folosit de utilitarul de generare a IP Core-urilor pentru a genera o instanță a core-ului selectat. Core-ul de tip Block Memory oferă o serie de parametri care pot fi configurați, precum dimensiunea cuvântului, numărul de cuvinte și semnalele disponibile. Block Memory oferă și posibilitatea inițializării memoriei în momentul generării. Conținutul memoriei poate fi specificat într-un fișier *.coe* referențiat apoi de fișierul *.xco*. În figura de mai jos puteți observa formatul fișierul *.coe*. Valorile din fișier sunt mapate în memoria generată începând de la adresa 0.

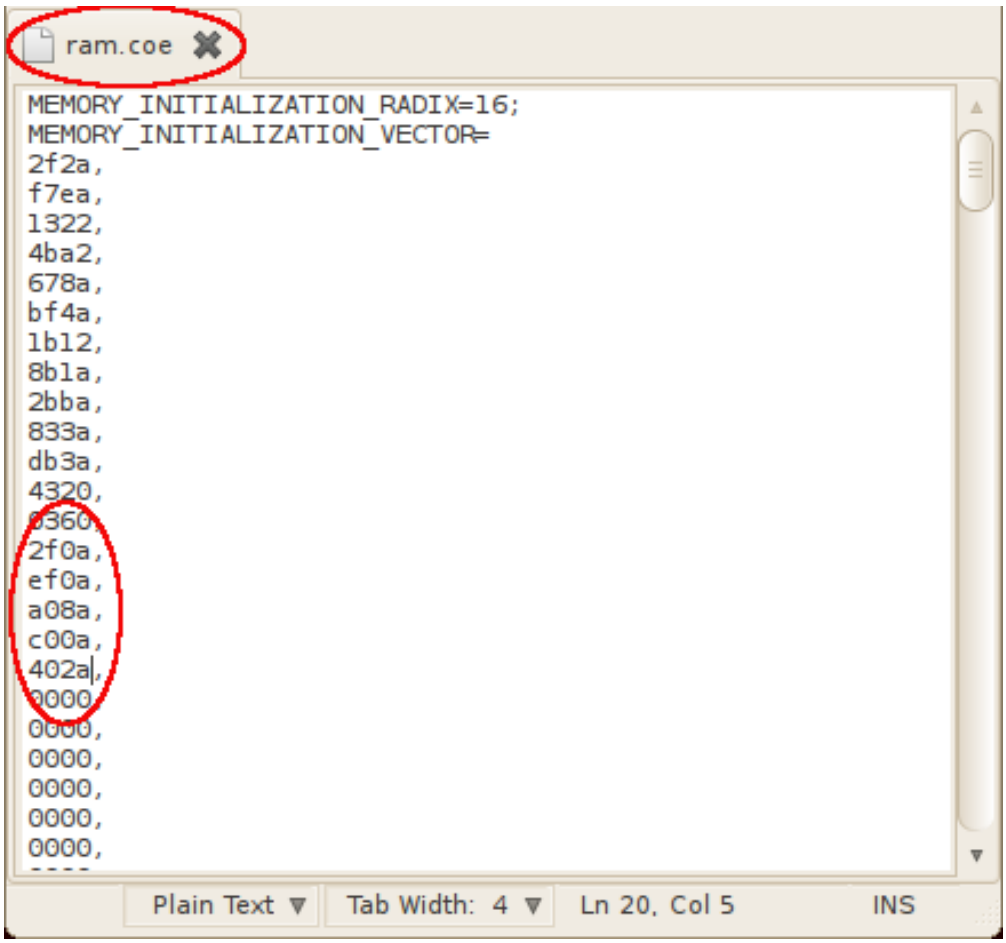


Figura 4: Fișierul ram.coe

Conținutul inițial al memoriei calculatorului didactic se găsește în cadrul acestui fișier. După fiecare modificare a fișierului *.coe*, memoria care folosește acel fișier trebuie regenerată. În cazul nostru, după modificarea fișierului *ram.coe* trebuie regenerat modulul *ram* pentru ca modificările să aibă efect. Pentru aceasta, se selectează din fereastra *Sources* modulul *ram*, iar din fereastra *Processes* se inițiază regenerarea, cu opțiunea *Regenerate Core*.